Guía de Gestión de Arrays, Strings y Parsing

Manipulación, búsqueda, transformación e inmutabilidad en JavaScript

1. REGLAS DE DECLARACIÓN: CONST VS LET

Declaración con `const`

Impide reasignar la referencia, pero **permite mutar** el contenido interno.

```
const arr = ["A"];
arr.push("B"); // ✅ Permitido
// arr = ["C"]; // ❌ Error
```

Declaración con `let`

Permite modificar el contenido y también **reasignar** la variable por completo.

```
let list = [1, 2];
list = list.filter(n => n > 1); // ✅ OK
```

2. MANIPULACIÓN DE CADENAS (STRINGS) Y PARSING

Transformaciones de Cadenas y Conversión de Tipos

Strings & Parsing

```
// Mayúsculas y Minúsculas
const texto = "Hola Mundo";
texto.toUpperCase(); // "HOLA MUNDO"
texto.toLowerCase(); // "hola mundo"

// Contar caracteres (Strings) o ítems (Arrays)
console.log(texto.length); // 10

// Convertir texto a número entero en base 10
const num = parseInt("42", 10); // 42
```

3. BÚSQUEDA Y VERIFICACIÓN EN ARRAYS

Índices, Presencia y Métodos Complejos

Búsquedas

```
// 1. Primer índice de un elemento simple
const letras = ["a", "b", "c"];
letras.indexOf("b"); // 1 (o -1 si no existe)

// 2. Índice en objetos / multidimensional
const users = [{ id: 1, nom: "Ana" }, { id: 2, nom: "Luis" }];
const idx = users.findIndex(u => u.id === 2); // 1

// 3. Obtener TODOS los índices coincidentes
const vals = ["a", "b", "c", "b"];
const allIdx = vals.map((v, i) => v === "b" ? i : -1).filter(i => i !== -1); // [1, 3]

// 4. Existencia global y condicionales
vals.includes("a"); // true
[10, 20].some(n => n > 15); // true
```

4. MODIFICACIÓN DIRECTA DEL ARRAY (MUTACIÓN IN-PLACE)

Insertar, Eliminar y Reemplazar

Mutables

```
// Operaciones en extremos: push/pop (final), unshift/shift (inicio)
const items = ["b"];
items.push("c"); // ["b", "c"]
items.unshift("a"); // ["a", "b", "c"]

// Modificación precisa en cualquier posición (splice)
// Estructura: array.splice(inicio, cantidadEliminar, elemento1, elemento2...)
const frutas = ["Manzana", "Pera", "Uva"];
frutas.splice(1, 1, "Plátano", "Kiwi"); // ["Manzana", "Plátano", "Kiwi", "Uva"]
```

5. CREACIÓN DE NUEVAS VERSIONES (INMUTABLES)

Transformaciones sin alterar el Array Original

Inmutables

```
// Filtrar (filter) y Transformar (map)
const nums = [1, 2, 3, 4];
const pares = nums.filter(n => n % 2 === 0); // [2, 4]
const dobles = nums.map(n => n * 2); // [2, 4, 6, 8]

// Copia y combinación (Spread ...)
const copia = [...nums];
const combinada = [0, ...nums, 5];

// ES2023+: Métodos inmutables directos
const ordenado = nums.toSorted();
const modificado = nums.with(0, 99); // Reemplaza idx 0 por 99
```